CS611 · MACHINE LEARNING ENGINEERING

Medallion Architecture Data Pipeline

End-to-End for a loan-risk model

Sun Jie-Sheng, Jason

Assignment 1 — May 2026

What this pipeline builds, and why

This pipeline ingests from raw CSV sources, cleans them through a Bronze → Silver → Gold flow, and assembles a training-ready dataset for a downstream machine-learning model. The model itself sits outside of this scope.

Engineering priorities

Reproducibility — one Docker container, one main.py, ensures consistent output

Modularity — small, single-purpose functions per layer, easy to test and replace

Auditability — every transformation traceable through bronze → silver → gold, and logged at runtime

Leakage-safety — features built only from data available at application time

Deliverables

Bronze, Silver and Gold datamart

Four source CSVs → partitioned by monthly snapshot date, written through three layers.

Gold label store + feature store

Two Gold tables ready to be consumed by an ML training pipeline.

Reproducible execution

docker-compose up + python main.py rebuilds the entire datamart from raw CSVs.

Raw sources from upstream systems

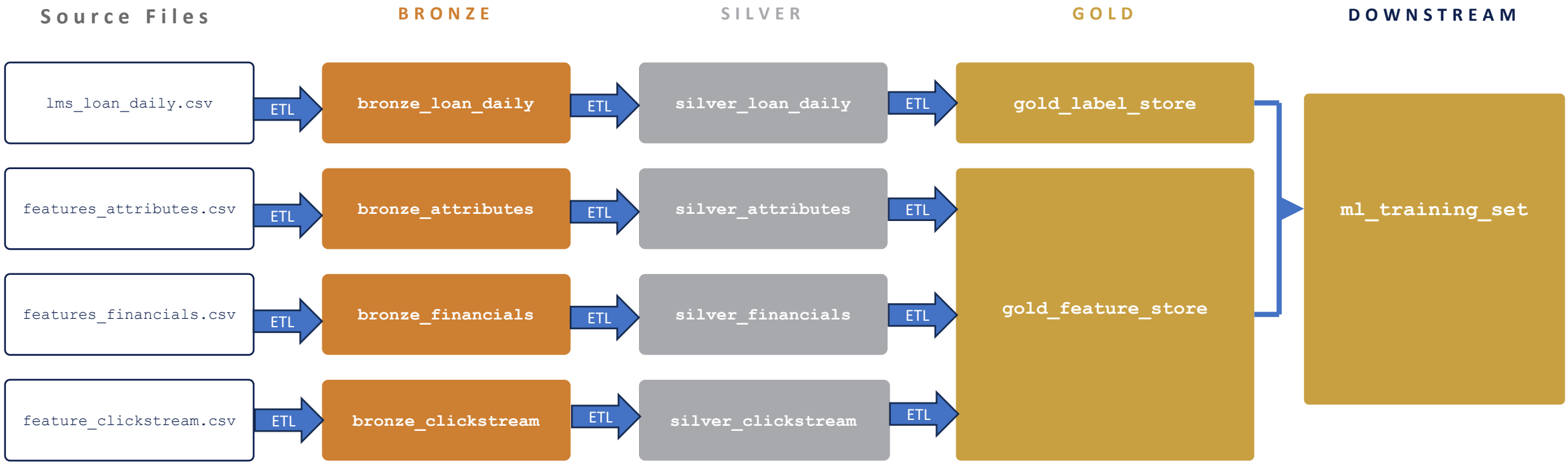
Source Files	Rows	Unique customers	Snapshot date range
lms_loan_daily.csv	137,500	12,500	2023-01 → 2025-11
features_attributes.csv	12,500	12,500	2023-01 → 2025-01
features_financials.csv	12,500	12,500	2023-01 → 2025-01
feature_clickstream.csv	215,376	8,974	2023-01 → 2024-12

Things to note

· Snapshot ranges differ across sources
· 3,526 customers are absent from clickstream entirely
· All loans in the LMS are uniform ($10k, 10-month tenure)
· Customer_ID is the only primary key across all four sources

Medallion Architecture

Pipeline Overview



Each layer is partitioned by monthly snapshot_date

Bronze Layer

What Bronze is for

A read-only copy of each source CSV, partitioned by month. The purpose of this layer is to faithfully preserve what arrived from the source systems, so we always have an original to compare against and a stable starting point for the pipeline.

Design choices

One process for every source · A step ingests all four CSVs with consistent file naming.

One file per source per month · Each month can be reprocessed independently.

Stored as CSV · Easy Readability for all.

Runtime logging · Every ingestion step provides a log for audit trail purposes.

utils/data_processing_bronze_table.py

```
def process_bronze_table(
    source_name,
    csv_path,
    date_str,
    bronze_dir,
    spark,
):
    df = (spark.read.csv(
        csv_path,
        header=True,
        inferSchema=True,
    ).filter(
        col('snapshot_date') == date
    ))
    print(f'[bronze {source_name}]'
        f' {date_str} rows: {df.count()}'
    )
    df.toPandas().to_csv(
        bronze_dir + name,
    )
```

Silver Layer - The Single Source of Truth

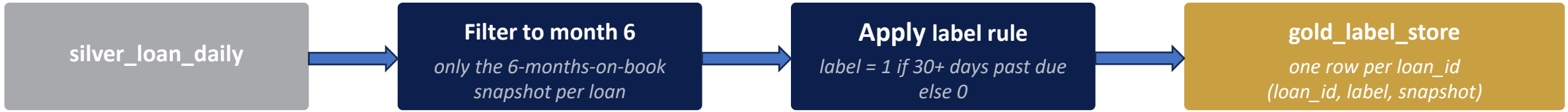
Silver is the clean version of each source. Per source, Silver reads the Bronze CSV, casts every column to its proper type, strips formatting dirt, replaces placeholder strings with NULL, caps numeric outliers, and parses semi-structured fields.

Column / Field	Data issue (from EDA)	What Silver does
Age	Impossible values (-500 placeholder, ages > 100); trailing underscore on some numbers	Strip underscore, cast to int, NULL for extreme values
14 financials numerical fields	Trailing underscore; extreme/impossible values (e.g. -100 loans, 4,293 missed payments, $23.8M income)	Strip underscore, cast to numeric, cap to plausible range
Occupation, Credit_Mix, Payment_of_Min_Amount, Payment_Behaviour	Placeholder strings (e.g. _____, #F%$D@*&8, _, NM, !@9#%8)	Replace placeholders with NULL
Credit_History_Age	English text (e.g. "10 Years and 9 Months")	Parse to total months as integer
Type_of_Loan	Comma-separated list with stray "and " on the last item	Parse into a clean array of loan-type strings.

Before → After examples

Age: "3843_" → NULL · **Annual_Income:** "52312.68_" → 52312.68 · **Credit_History_Age:** "10 Years and 9 Months" → 129
Credit_Mix: "_" → NULL · **Type_of_Loan:** "Credit-Builder Loan, and Home Equity Loan" → ["Credit-Builder Loan", "Home Equity Loan"]

Gold Layer – Label Engineering



Design decisions

Why month 6 on book? Industry-standard early-default observation window.

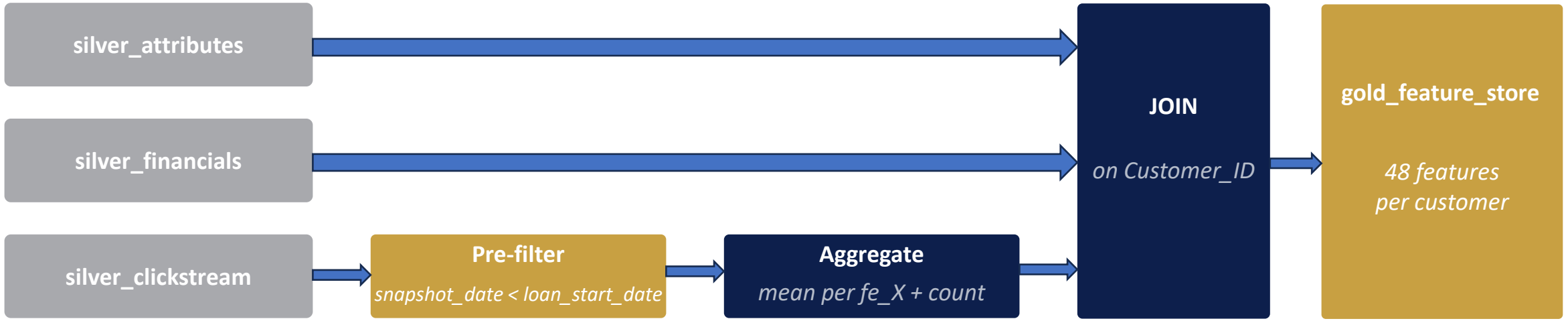
Why '30 days past due' as the threshold? A standard delinquency definition used across consumer lending.

Today = the latest cross-source snapshot. The latest snapshot available across all four sources. Customers whose MOB 6 falls after today are not yet labelable. As new snapshots arrive, the pipeline will label them.

Result (today = 2024-12-01)

Metric	Value
Labelled loans (training set)	**8,974**
Non-default (label = 0)	**6,383**
Default (label = 1)	**2,591**
Default rate	**29%**
In-flight (no MOB 6 yet)	**3,526**
Monthly Partitions	**18**

Gold Layer - Feature Engineering



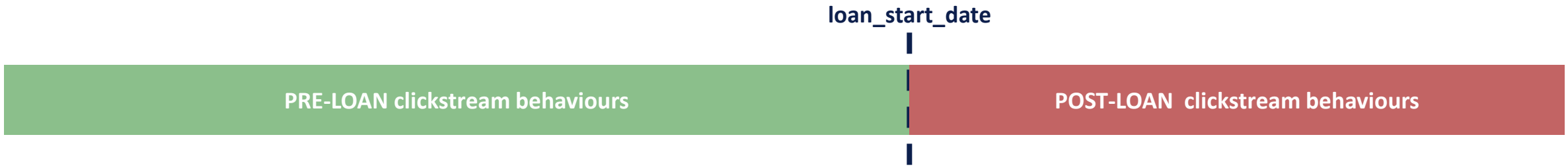
Design decisions

Why not impute missing values here?

Where data is missing or cleaned to NULL upstream, it is also retained in the gold layer. Tree-based models handle NULLs natively, and linear models can impute deterministically at training time. Imputing in the feature store would force one strategy on every downstream models.

Clickstream Temporal Leakage

Each customer's 24 monthly clickstream snapshots span both **before** and **after** their loan-start date. Simply averaging all 24 mixes pre and post loan actions into a feature that does not exist at the point of a new loan application, this is an example of temporal leakage.



Example: Alice (defaulted at MOB 6)

Window	Avg fe_1
Pre-loan (14 months)	**~3**
Post-loan (10 months — panic)	**~14**
Average across all 24 months	**~7.6**
Pre-filter avg	**~3**

The filter

```
ck_pre_loan = ck_joined.filter(
    col('snapshot_date')
    < col('loan_start_date')
)
```

Why it matters. A new applicant at prediction time only has pre-loan clickstream. Training on the post-loan averages teaches the model patterns production data structurally cannot reproduce.

Pipeline Summary

What gets produced. Across monthly snapshots source files, the datamart yields three Gold tables -> a label store, a feature store, and an ml_training_set ready for downstream usage. The pipeline labels 8,974 matured customers and identifies 3,526 in-flight customers awaiting their MOB 6 snapshot.

Where each layer earns its keep. Bronze preserves the source-system drop exactly as it arrived. Silver provides the single source of truth. Gold stores the engineered datasets, applies the label rule, and filtered clickstream so features cannot include post-application behaviour.

What the Medallion Architecture Pipeline provide us? Reproducibility -> Docker provides the consistent environment and PySpark provides us with the toolkit with identical output every run. Auditability -> every transformation is traceable through the bronze→silver→gold trail and printed to the runtime log. Modularity -> one utils function per layer per source, easy to edit and make changes to the pipeline.

What is in datamart/

datamart/
- bronze/
  - loan_daily/ .... 24 CSV
  - clickstream/ ... 24 CSV
  - attributes/ .... 24 CSV
  - financials/ .... 24 CSV
- silver/
  - loan_daily/ .... 24 parquet
  - clickstream/ ... 24 parquet
  - attributes/ .... 24 parquet
  - financials/ .... 24 parquet
- gold/
  - **label_store/ ...**
  - **feature_store/ ...**
  - **ml_training_set/**